

Every `StringBuffer` starts off with 16 characters of capacity. When you initialize it with a `String`, you still get the original 16, and your initializing `String` is just added to that 16. If you add characters to your `StringBuffer`, it grows in size, it's Dynamically Resizable. You don't worry about its size, it's a question of efficiency.

For example, this `StringBuffer` would be inefficient:

```
StringBuffer e;  
e = new StringBuffer( 99999999 );
```

This one will hold 99999999 characters. If you do wish to add to your `StringBuffer`, you use its `append()` method

```
StringBuffer e;  
e = new StringBuffer( 100 );  
e.append( "Key Largo" );
```

A `StringBuffer` makes your code run faster because you create fewer new objects such as temporary `Strings`. The method `append()` always adds the characters to then end of the `StringBuffer`. The method `append()` is overloaded 10 times. `append()` takes arguments of every primitive data type and will convert each to a `String` automatically. You cannot intermingle `String` methods with `StringBuffer` methods.

When the overloaded plus sign `[ + ]` concatenates a `String` to something else, it uses a `StringBuffer` object:

```
d = "Ten" + 4 + "ya";
```

The above is actually implemented as follows:

```
d=new StringBuffer().append("Ten").append(4) .  
append("ya");
```

This is initially created with the default capacity.

Method `append()` is the principle way to add characters to the end of a `StringBuffer`. `StringBuffer` method `insert()` is used to place characters into an existing `StringBuffer` object at a specified location. Method `insert()` is overloaded 9 times.

Whereas `append()` adds text to the end of the `StringBuffer` object, method `insert()` takes two arguments. The first argument is an integer "offset" that tells the first character position to begin inserting the second argument. The second argument can be any data type.

Classes `String` and `StringBuffer` do share a few methods. You can learn the length of a `StringBuffer` by using the method: `length()`. Method `length()` tells you the amount of memory a `StringBuffer` currently is using. Method